CTU training solutions | Private Tertiary Education

CAPSTONE PROJECT

DATABASE DESIGN DOCUMENT

Student / Group Template

Project Title	SmartDesk: A Customer Support Ticketing System for Small IT Support Teams
Group Members	Freek Schaefer-Gerber and Wian Sandenbergh
Module	SPM622 / MDB622
Lecturer / Supervisor	Mr. Dunstan
Week / Stage	Design finalised
Submission Date	29 October 2026

CTU Training Solutions (PTY) Ltd is fully accredited by the MICT SETA (ACC/2006/07/186) through the Quality Council for Trades and Occupations (QCTO). CTU Training Solutions is registered with the Department of Higher Education and Training as a Private College (2018/FE07/004) and provisionally registered as a private higher education institution (No 2014/HE07/004) Co. Reg No:2005/038944/07

Database Purpose

Explain what information the database must support.

The SmartDesk database must support four functions:

1. Authentication and authorisation. Store user accounts with a one-way password hash and a role (`customer`, `agent`, `admin`) that determines what each user may do.
2. Ticket management. Record support requests with a classification (`category`), an urgency (`priority`), a lifecycle state (`status`), the customer who raised it, and the staff member assigned to it (if any).
3. Attributable conversation. Allow both the customer and staff to add comments to a ticket, with each comment attributed to its author and timestamped.
4. Scoped retrieval. Support dashboard counts by status and ticket lists that a customer can only see their own records from.

The database must enforce referential integrity — a comment cannot exist without a ticket, and a ticket cannot exist without a creating customer — and must never store a recoverable password.

Scope note. The running application uses SQLite. A parallel Microsoft SQL Server script (`sql/sqlserver_schema.sql`) is supplied as a separate MDB622 database-design artefact and is not connected to the application. This document describes the runtime SQLite schema; the SQL Server equivalent is addressed under SQL Scripts.

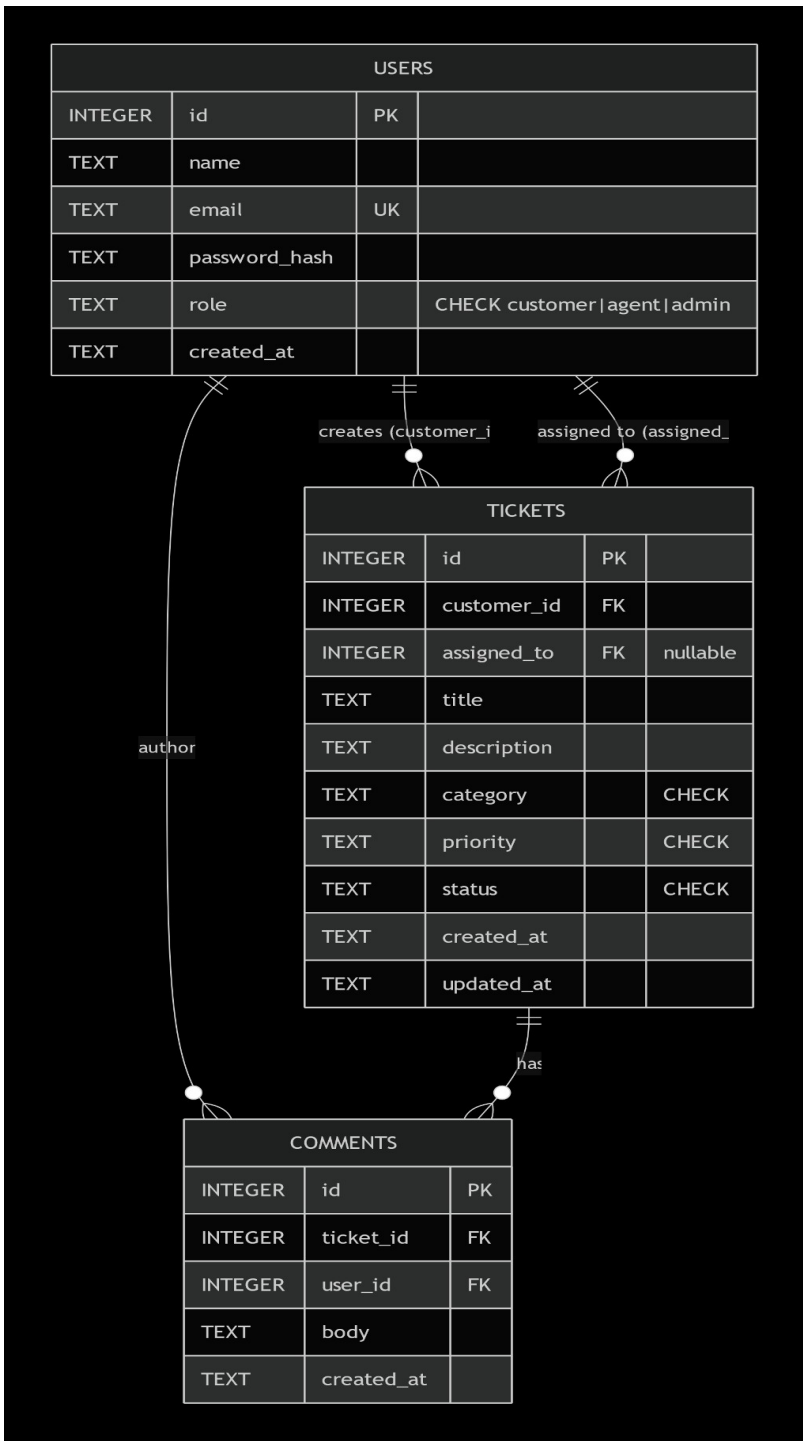
Entity-Relationship Diagram

Insert the ERD and explain cardinalities.

Figure 1 — Entity-Relationship Diagram of the SmartDesk runtime database.

(Render the Mermaid code below at mermaid.live, export as PNG, and insert the image. Do not paste raw code into the dissertation.)

CTU Training Solutions (PTY) Ltd is fully accredited by the MICT SETA (ACC/2006/07/186) through the Quality Council for Trades and Occupations (QCTO). CTU Training Solutions is registered with the Department of Higher Education and Training as a Private College (2018/FE07/004) and provisionally registered as a private higher education institution (No 2014/HE07/004) Co. Reg No:2005/038944/07



CTU Training Solutions (PTY) Ltd is fully accredited by the MICT SETA (ACC/2006/07/186) through the Quality Council for Trades and Occupations (QCTO). CTU Training Solutions is registered with the Department of Higher Education and Training as a Private College (2018/FE07/004) and provisionally registered as a private higher education institution (No 2014/HE07/004) Co. Reg No:2005/038944/07

Cardinalities

Relationship	Cardinality	Participation	Implementation
`users` → `tickets.customer_id`	**1 : N**	Mandatory on the ticket side	`customer_id INTEGER NOT NULL REFERENCES users(id)`
`users` → `tickets.assigned_to`	**1 : N**	**Optional** on the ticket side	`assigned_to INTEGER REFERENCES users(id)` — nullable; `NULL` means unassigned
`tickets` → `comments`	**1 : N**	Mandatory on the comment side	`ticket_id INTEGER NOT NULL REFERENCES tickets(id) ON DELETE CASCADE`
`users` → `comments`	**1 : N**	Mandatory on the comment side	`user_id INTEGER NOT NULL REFERENCES users(id)`

Reading the model. A user may create many tickets but each ticket has exactly one creator. A staff user may be assigned many tickets, but a ticket has zero or one assignee — hence the nullable foreign key rather than a junction table. A ticket accumulates many comments, and deleting a ticket cascades to remove its comments, since a comment has no meaning without its ticket. Deleting a user is not cascaded: the default restrict behaviour prevents deleting a user who is referenced by any ticket or comment, which preserves ticket history.

Data Dictionary

For each table, document column name, data type, size, nullability, key and description.

Note on `TEXT` columns. SQLite `TEXT` has no declared maximum length; length limits are enforced in application code by truncation on input (e.g. `title[:150]`). This is recorded as a limitation under Security and Integrity. Sizes shown in the App limit column indicate the enforced application-level maximum, not a database constraint.

Table: `users`

Stores all accounts — customers, agents and administrators — distinguished by `role`.

Column	Data type	App limit	Nullable	Key	Description
`id`	INTEGER	—	No	**PK**, AUTOINCREMENT	Surrogate identifier
`name`	TEXT	100	No		Display name shown on tickets and comments
`email`	TEXT	200	No	**UNIQUE**	Login identifier; lower-cased and trimmed on entry
`password_hash`	TEXT	255	No		Werkzeug `generate_password_hash` output (scrypt). Never the plain password
`role`	TEXT	—	No		`CHECK (role IN ('customer','agent','admin'))`, default `'customer'`
`created_at`	TEXT	—	No		UTC timestamp, `YYYY-MM-DD HH:MM:SS`

Table: `tickets`

Stores one row per support request.

Column	Data type	App limit	Nullable	Key	Description
`id`	INTEGER	—	No	**PK**, AUTOINCREMENT	Ticket number, displayed as `#0001`
`customer_id`	INTEGER	—	No	**FK →** `users.id`	The customer who raised the ticket
`assigned_to`	INTEGER	—	**Yes**	**FK →** `users.id`	Assigned staff member; `NULL` = unassigned. Validated in code to hold `agent`/`admin` only
`title`	TEXT	150	No		Short subject line
`description`	TEXT	5000	No		Full description of the issue

Column	Data type	App limit	Nullable	Key	Description
`category`	TEXT	—	No		`CHECK (category IN ('Account','Billing','Technical','Product','General'))`
`priority`	TEXT	—	No		`CHECK (priority IN ('Low','Medium','High','Critical'))`
`status`	TEXT	—	No		`CHECK (status IN ('Open','In Progress','Pending','Resolved','Closed'))`; set to `'Open'` on creation
`created_at`	TEXT	—	No		UTC timestamp set by the application
`updated_at`	TEXT	—	No		UTC timestamp; refreshed on ticket update and on new comment

Table: `comments`

Stores the attributable conversation attached to each ticket.

Column	Data type	App limit	Nullable	Key	Description
`id`	INTEGER	—	No	**PK**, AUTOINCREMENT	Surrogate identifier
`ticket_id`	INTEGER	—	No	**FK** → `tickets.id`, `ON DELETE CASCADE`	Parent ticket
`user_id`	INTEGER	—	No	**FK** → `users.id`	Author of the comment
`body`	TEXT	3000	No		Comment text
`created_at`	TEXT	—	No		UTC timestamp; ordering key for the conversation

Normalisation

Explain how duplication and update anomalies were reduced.

1NF. Every table has a primary key and holds only atomic values. The conversation is not a text field appended to on `tickets`; it is decomposed into `comments`, one row per message with its own author and timestamp.

2NF. All tables use a single-column surrogate key (`id`), so no non-key attribute can depend on part of a composite key.

3NF. No non-key attribute determines another. `comments` stores `user_id` rather than the author's name, so a name change is a single-row update in `users` and every comment reflects it. `tickets` stores `customer_id` and `assigned_to` rather than names or emails, for the same reason.

Deliberate departure. `category`, `priority` and `status` are `CHECK`-constrained `TEXT` rather than lookup tables. This does not violate 3NF — the value depends only on the ticket's primary key, and the domain is enforced by a constraint. It is a denormalisation of reference data with a known cost: adding a category requires a schema change (`ALTER TABLE` and a new `CHECK`) rather than an `INSERT`. Accepted for MVP simplicity and recorded as a limitation in Chapter 6.

Security and Integrity

Explain constraints, validation, access control, backups and sensitive data handling.

Integrity. Primary keys on all tables. `REFERENCES` on all relationships, with enforcement explicitly switched on via `PRAGMA foreign_keys=ON` in `db()` — SQLite ignores foreign keys by default, so without this they would be advisory only. `comments.ticket_id` cascades on delete; all other foreign keys restrict, so a user referenced by any ticket or comment cannot be deleted. `CHECK` constraints on `role`, `category`, `priority`, `status`; `UNIQUE` on `users.email`.

Validation. Registration checks name, @ in email, password ≥ 8 chars, and catches duplicate email as `IntegrityError`. Ticket creation checks non-empty title and description, and validates `priority` and `category` against module tuples (allow-list). Ticket update validates `status` and `priority` the same way, and re-checks in SQL that the assignee holds role `agent` or `admin`, since a client can post any value. All strings are truncated to column limits on entry — this, not the database, enforces the "App limit" values in the data dictionary.

Access control.

Control	Mechanism
Authentication	Session-based; `login_required` on every route except `/login` and `/register`
Credentials	Werkzeug `check_password_hash` against the stored hash
Session	`session.clear()` before assigning `user_id`, mitigating fixation

Control	Mechanism
Role	`update_ticket` aborts 403 unless role is `agent` or `admin` — the route check, not the hidden UI panel, is the control
Ownership	`get_ticket()` aborts 403 on cross-customer access; all ticket-reading routes call it, so it cannot be bypassed
Scoping	Dashboard adds `WHERE customer_id = ?` for customers
CSRF	`before_request` rejects mismatched `_csrf` on any POST, using `secrets.compare_digest`
Injection	Parameter substitution (?) throughout; no string formatting with user input

Sensitive data. Passwords stored only as Werkzeug hashes, never logged. Login errors are identical for unknown email and wrong password, so account existence is not disclosed. All demo data is synthetic. SECRET_KEY and DATABASE_PATH come from environment variables; no credentials in Git.

Backups. SQLite is a single file (`support.db`). Backup is a manual copy with the app stopped; `flask init-db` and `flask seed-demo` rebuild from scratch. No automated backup or retention policy.

Known limitations. (1) No database-level users or permissions. (2) No database-level length enforcement. (3) No audit table — only current status is stored. (4) No enforced status transitions. (5) Development secret-key fallback in `app.py`. (6) No rate limiting on `/login`. (7) No password reset. (8) Timestamps stored as TEXT.

SQL Scripts

Reference the schema and seed scripts stored in the project repository.

Script	Location	Purpose
Runtime schema	`sql/sqlite_schema.sql`	Creates the three tables and indexes; idempotent. Run by `flask init-db`
Seed data	`flask seed-demo` in `app.py`	Three demo accounts and one ticket; idempotent

Script	Location	Purpose
SQL Server schema	`sql/sqlserver_schema.sql`	Equivalent SQL Server 2022 design for MDB622; run manually in SSMS

Reproduction. Delete `support.db` → `flask init-db` → `flask seed-demo` → `python app.py`.

Statement of fact. The application connects to SQLite only. `sqlserver_schema.sql` is a parallel design artefact, not the runtime database. Migration would require replacing the `sqlite3` layer and the `PRAGMA` statements.

CTU Training Solutions (PTY) Ltd is fully accredited by the MICT SETA (ACC/2006/07/186) through the Quality Council for Trades and Occupations (QCTO). CTU Training Solutions is registered with the Department of Higher Education and Training as a Private College (2018/FE07/004) and provisionally registered as a private higher education institution (No 2014/HE07/004) Co. Reg No:2005/038944/07